

AI Design Document

Reorder Suggestion Engine & Training Risk Flag · Requirement WST-FR-14

Field	Value
Author / Role	AI Engineer — Squad 6
Requirement	WST-FR-14 (Priority: Should — implemented after all Must items)
Scope	Rule-based baseline first; optional model only after sufficient clean history
Status	Draft v1.0

1. Purpose & Scope

This document defines the design of the two AI/data features required by the project brief under WST-FR-14: the Reorder Suggestion engine and the Training Risk Flag engine. Both features are delivered as an explainable, rule-based baseline first. A statistical or ML model is an optional, later addition and must be evaluated against this baseline — it is never a prerequisite for the MVP.

Per the project's five shipping rules: every AI feature needs a deterministic baseline and a fallback, and a model that cannot beat its baseline is a valid, reportable result.

2. Feature 1 — Reorder Suggestion Engine

2.1 Inputs

Field	Source	Description
Part_ID	Part	Unique identifier of the spare part
Current_Stock	Part / StockMovement	On-hand quantity in store
Reserved_Quantity	StockMovement	Quantity already reserved against open job cards
Min_Stock / Max_Stock	Part	Configured reorder thresholds
Open_PO_Quantity	PurchaseOrder	Quantity already on order but not yet received
Avg_Weekly_Consumption	StockMovement (rolling window)	Recent average weekly usage, e.g. trailing 4–8 weeks

2.2 Processing Logic

Step 1 — Compute Available Stock:

```
Available_Stock = Current_Stock - Reserved_Quantity
```

Step 2 — Compute Net Position (accounts for stock already inbound, avoiding duplicate orders):

```
Net_Position = Available_Stock + Open_PO_Quantity
```

Step 3 — Trigger condition:

```
if Net_Position < Min_Stock:
    trigger reorder
```

Step 4 — Suggested quantity (brings stock up to Max, minus what is already inbound):

```
Suggested_Quantity = max(0, Max_Stock - Net_Position)
```

Step 5 — Risk level classification, using consumption velocity relative to what's on hand:

```
Weeks_of_Cover = Available_Stock / Avg_Weekly_Consumption    (guard: div by 0 -> treat as High)
if Weeks_of_Cover < 1:      risk = "High"
elif Weeks_of_Cover < 3:    risk = "Medium"
else:                      risk = "Low"
```

2.3 Output

```
{
  "part_id": "P-0042",
  "part_name": "Brake Pad",
  "suggested_quantity": 20,
  "risk": "High",
  "reason": "Available stock covers 0.6 weeks of demand; below minimum threshold",
  "baseline_version": "rule-v1.0",
  "model_version": null,
  "generated_at": "2026-09-18T10:00:00Z"
}
```

2.4 Rules Summary

- A purchase order is never placed automatically — the system only produces a suggestion (per Acceptance Scenario #3 in the brief).
- Open POs are always subtracted from need, to prevent duplicate ordering.
- Thresholds (Min/Max) are configured per part by the storekeeper, not hard-coded.
- Division-by-zero / missing consumption history defaults to High risk with an explicit "insufficient history" flag rather than a silent error.

3. Feature 2 — Training Risk Flag Engine

3.1 Inputs

Field	Source	Description
Student_ID	Student	Unique student identifier
Attendance_Rate	Attendance	% sessions attended vs scheduled
Unsigned_Assessments	Assessment	Count of results pending supervisor sign-off
Missing_Competencies	Competency	Count of required competencies not yet met
Time_On_Task	Assessment / PracticalTask	Actual vs expected task duration

3.2 Processing Logic — Weighted Rule Score

Each factor contributes points toward a 0–100 risk score. Weights are configuration, not hard-coded, so the training supervisor can tune them.

```

score = 0
if Attendance_Rate < 60%:      score += 40
elif Attendance_Rate < 80%:    score += 20

if Unsigned_Assessments >= 3:  score += 25
elif Unsigned_Assessments >= 1: score += 10

if Missing_Competencies >= 2:  score += 25
elif Missing_Competencies == 1: score += 10

if Time_On_Task < 70% of expected: score += 10

risk = "High"   if score >= 50
risk = "Medium" if 25 <= score < 50
risk = "Low"    if score < 25

```

3.3 Output

```

{
  "student_id": "S-0123",
  "risk": "High",
  "score": 65,
  "reason": "Attendance below 60%; 2 competencies missing",
  "contributing_factors": ["low_attendance", "missing_competencies"],
  "baseline_version": "rule-v1.0",
  "model_version": null,
  "generated_at": "2026-09-18T10:00:00Z"
}

```

3.4 Rules Summary

- Risk flags are advisory only — the brief explicitly excludes automatic grades, certification, or safety decisions made solely by AI.
- A flag never blocks or alters a sign-off; supervisors keep full authority over assessment results.
- If a future logistic-regression model is trained on this data, it must use learning-activity fields only — never protected attributes (per the brief's AI/data-science scope).

4. Explainability & the Prediction Log

Every suggestion produced by either engine — rule-based or, later, model-based — is written to a Prediction record so it is auditable and traceable, as required by the brief's acceptance evidence for WST-FR-14.

Field	Description
inputs	Snapshot of all input values used for this prediction
baseline_version / model_version	Which logic produced the result (e.g. rule-v1.0, or a trained model tag)
explanation	Human-readable reason string, shown in the UI
user_decision	accepted / overridden / ignored, plus the overriding user
evaluation_outcome	Filled in later once the real-world result is known, for metric tracking

5. Fallback Behavior

- The AI service (FastAPI, optional Python process) runs independently of core workshop/training operations — job cards, stock issuing, attendance, and sign-offs must work with the AI service fully offline (Acceptance Scenario #8).
- If the service is unreachable, the API returns a defined fallback response (e.g. "prediction unavailable") rather than blocking the caller; the Backend treats this as a soft-fail, not an error.
- Reorder and risk data continue to display using the last successfully cached prediction, clearly marked as stale, if configured.

6. APIs Exposed to Backend / Frontend

POST /ai/reorder

Input: part-level stock and consumption fields (Section 2.1). Output: Section 2.3 JSON.

POST /ai/training-risk

Input: student-level attendance/assessment fields (Section 3.1). Output: Section 3.3 JSON.

GET /ai/reorder-alerts

Returns the current list of parts flagged Medium/High, for the dashboard — no model internals exposed, per the brief's rule that the Frontend does not need to know how the prediction was made.

7. Metrics (per brief's AI/data measurement set)

Metric	Applies to	Purpose
Reorder acceptance / override rate	Reorder engine	How often storekeepers trust the suggestion
Forecast MAE / WAPE	Reorder engine (once a model exists)	Compare model accuracy vs moving-average baseline
Risk precision / recall	Training risk engine	Correctness of High-risk flags against real outcomes
Missing / late data rate	Both	Data-quality signal that limits when a model is safe to trust

8. Delivery Plan (implementation reference)

- Week 1 — this design document, finalized inputs/outputs, shared with Backend and Data/BI.
- Week 2 — Rule Engine implementation for both features (this is the Should-priority MVP baseline).
- Week 3 — FastAPI service exposing the three endpoints above, with Swagger documentation.
- Week 4 (if time allows) — Logistic regression for training risk / moving-average-vs-model comparison for reorder demand, evaluated strictly against the Week 2 baseline.